

Detailed Per-Person Job Breakdown (Parallel Work)

PERSON 1: RENDERING & GRAPHICS ENGINEER

Role: Visual output and camera system

Dependencies Needed:

- GLFW (windowing)
- GLAD (OpenGL loader)
- GLM (math library)
- Basic understanding of OpenGL pipeline
- Access to shared **Agent** struct definition

Input Requirements from Others:

- **From Person 2:** Pointer to GPU buffer containing agent positions and velocities
- **From Person 4:** UI parameters don't affect rendering directly (mostly independent)

Output Deliverables:

- Rendering system that can draw N agents efficiently
- Camera controls for navigation
- Visual effects and color schemes

Detailed Tasks:

Task 1.1: OpenGL Context & Window Setup

- Initialize GLFW window (800x600 or configurable)
- Create OpenGL context (version 4.6 core profile)
- Set up viewport and clear color
- Implement basic event handling (keyboard, mouse)
- Create main render loop structure
- Handle window resize events

Task 1.2: Camera System (2D with 2.5D capability)

- Implement 2D orthographic camera initially
 - Pan: Mouse drag to move view
 - Zoom: Mouse scroll to zoom in/out
 - Calculate view matrix from camera position and zoom
- Store camera state (position, zoom level, rotation)

- Add keyboard controls (WASD for pan, Q/E for rotation)
- Later: Upgrade to perspective camera for 2.5D effect
 - Calculate perspective projection matrix
 - Allow camera tilt/orbit around the swarm

Task 1.3: Shader Programs

- Write vertex shader:
 - Take base geometry (triangle vertices)
 - Take per-instance data (position, velocity from buffer)
 - Transform vertices based on instance position
 - Rotate instance to align with velocity direction
 - Apply camera view and projection matrices
- Write fragment shader:
 - Calculate color based on velocity magnitude (heat map)
 - Support different color schemes (settable via uniform)
 - Add optional glow effect for specific agent types
- Compile and link shader programs
- Handle shader compilation errors gracefully

Task 1.4: Geometry Setup

- Create base agent geometry (single triangle/arrow shape)
 - Define vertices in model space
 - Create Vertex Array Object (VAO)
 - Create Vertex Buffer Object (VBO) for base geometry
- Set up vertex attributes for base geometry
- Design agent shape to look good when rotated

Task 1.5: Instanced Rendering System

- Create VBO for instance data (positions, velocities)
 - This will be shared with CUDA (Person 2 writes to it)
 - Set up as dynamic buffer
- Configure vertex attributes for instanced data
 - Position attribute (divisor = 1)
 - Velocity attribute (divisor = 1)
- Implement `glDrawArraysInstanced()` call
- Verify single draw call renders all agents

Task 1.6: Visual Effects

- **Color schemes:**
 - Velocity-based gradient (blue=slow, red=fast)
 - Type-based (prey=green, predator=red)
 - Uniform color option
 - Rainbow/psychedelic mode

- **Motion trails:**
 - Store history buffer of past positions
 - Render trails as lines or fading points
 - Configurable trail length
- **Glow effects:**
 - Additive blending for predators
 - Particle-like appearance
- **Background:**
 - Grid overlay option
 - Gradient background
 - Depth fog for 2.5D

Task 1.7: Performance Optimization

- Implement frustum culling (don't render off-screen agents)
- Level of Detail (LOD) system:
 - Close: Full triangle geometry
 - Medium: Smaller triangles
 - Far: Single points
- Minimize state changes per frame
- Profile rendering time separately from simulation

Task 1.8: Debug Visualization

- Option to render spatial grid cells (from Person 3)
- Show velocity vectors as lines
- Highlight specific agents for debugging
- Display coordinate axes

Task 1.9: 2.5D Upgrade

- Switch to perspective projection
- Position camera looking at 2D plane from angle
- Add depth testing
- Make agents billboard toward camera
- Smooth camera transitions

Files Person 1 Owns:

```
src/renderer.cpp/h
src/camera.cpp/h
shaders/agent.vert
shaders/agent.frag
shaders/trail.vert
shaders/trail.frag
```

Integration Points:

- Must coordinate with Person 2 on shared VBO format
 - Receives render calls from main loop
 - Exposes void `render(int agent_count)` function
-

PERSON 2: CUDA SIMULATION ENGINEER

Role: Core physics simulation on GPU

Dependencies Needed:

- CUDA Toolkit
- GLM (for device-side math)
- CUDA-OpenGL interop knowledge
- Access to shared `Agent` struct

Input Requirements from Others:

- **From Person 3:** Spatial hash data structure (for optimization phase)
- **From Person 4:** Simulation parameters (weights, speeds, etc.)

Output Deliverables:

- GPU kernels that update agent positions and velocities
- Shared GPU buffer that Person 1 can render
- Verified correct flocking behavior

Detailed Tasks:**Task 2.1: Agent Data Structure Design**

- Define `Agent` struct with proper alignment for GPU
 - Position (`vec2`)
 - Velocity (`vec2`)
 - Acceleration (`vec2`) - working variable
 - Max speed, max force, perception radius
 - Type (prey/predator) - `uint8_t` or enum
 - Energy (for predator-prey)
- Consider Structure of Arrays (SoA) vs Array of Structures (AoS)
- Ensure 16-byte alignment for coalesced memory access

Task 2.2: GPU Memory Management

- Allocate CUDA device memory for agent arrays
- Implement double-buffering (read from one, write to other)
- Create utility functions:

- Initialize agents with random positions/velocities
- Copy data to/from host if needed
- Free memory properly
- Handle dynamic agent count changes

Task 2.3: OpenGL-CUDA Interop

- Register OpenGL VBO with CUDA
- Map/unmap VBO for CUDA access
- Write directly to shared buffer (no CPU copy)
- Handle synchronization between OpenGL and CUDA
- Ensure proper resource cleanup

Task 2.4: Boids Algorithm - Naive Version ($O(n^2)$) Write CUDA kernel for agent updates: - **Kernel structure:** - Each thread handles one agent - Calculate grid/block dimensions - Handle boundary cases (thread index out of range)

- **For each agent, compute steering forces:**
 - **Separation:** Loop through ALL other agents
 - * If within personal space radius
 - * Calculate repulsion vector (inversely proportional to distance)
 - * Accumulate separation force
 - **Alignment:** Loop through ALL other agents
 - * If within perception radius
 - * Accumulate their velocities
 - * Average and use as desired velocity
 - **Cohesion:** Loop through ALL other agents
 - * If within perception radius
 - * Accumulate their positions
 - * Calculate center of mass
 - * Steer toward center
 - **Combine forces** with weights from parameters
- **Apply forces:**
 - Acceleration = total steering force (clamped to max_force)
 - Velocity += acceleration * dt
 - Clamp velocity to max_speed
 - Position += velocity * dt
- **Boundary conditions:**
 - Wrap-around (toroidal world): if $x > \text{width}$, $x -= \text{width}$
 - Or bounce off edges
 - Ensure agents stay in valid space

Task 2.5: Kernel Optimization (Pre-Spatial Hash)

- Minimize thread divergence in conditionals
- Use shared memory for frequently accessed data

- Coalesce memory accesses where possible
- Profile with NVIDIA Nsight Compute
- Tune block size (try 128, 256, 512 threads per block)

Task 2.6: Integration with Spatial Hash (from Person 3)

- Replace $O(n^2)$ neighbor search with hash-based queries
- For each agent:
 - Query only neighboring grid cells (9 cells in 2D)
 - Check only agents in those cells
 - Still apply same boids rules
- Verify behavior matches naive version

Task 2.7: Time Integration

- Implement proper time stepping (dt parameter)
- Support variable time step or fixed time step
- Ensure simulation stability at different frame rates
- Add simulation pause/resume

Task 2.8: Advanced Behaviors (Coordinate with Person 4)

- **Predator behavior:**
 - Different kernel or conditional logic
 - Pursuit steering toward nearest prey
 - Faster max speed than prey
- **Prey fear response:**
 - Extra separation force from predators
 - Weighted more heavily than normal separation
 - Panic mode when predator very close
- **Obstacle avoidance:**
 - Ray-cast or sphere-cast ahead
 - Calculate avoidance force perpendicular to obstacle
 - Blend with other steering forces

Task 2.9: Verification & Testing

- Implement CPU version of same algorithm
- Compare GPU results vs CPU for correctness
 - Same initial conditions should give same results
 - Check edge cases (1 agent, 2 agents, boundaries)
- Unit tests for steering force calculations
- Verify conservation of agent count

Task 2.10: Performance Profiling

- Measure kernel execution time

- Track memory bandwidth usage
- Identify bottlenecks (compute vs memory bound)
- Test scaling: 1K, 10K, 100K, 1M agents
- Document performance on different GPUs (MX130, 3050, 4060)

Files Person 2 Owns:

```
src/simulation.cu/h
src/agent.h (shared with others)
src/cuda_utils.cu/h
src/kernels.cu/h
```

Integration Points:

- Shares VBO with Person 1 (must agree on format)
- Uses spatial hash from Person 3
- Receives parameters from Person 4
- Exposes void `stepSimulation(float dt)` function

PERSON 3: SPATIAL OPTIMIZATION ENGINEER

Role: High-performance neighbor search system

Dependencies Needed:

- CUDA Toolkit
- Thrust library (comes with CUDA)
- Understanding of hash tables and sorting
- Access to `Agent` struct

Input Requirements from Others:

- **From Person 2:** Agent position data
- Grid cell size = perception radius (from parameters)

Output Deliverables:

- Spatial hash data structure
- Fast neighbor query function
- Documented performance improvements

Detailed Tasks:

Task 3.1: Spatial Hash Theory & Design

- Research 2D uniform grid spatial hashing
- Design hash table structure:

- Cell start/end arrays
- Sorted agent indices
- Agent-to-cell mappings
- Calculate optimal table size (typically ~2x agent count)
- Choose hash function for 2D grid cells

Task 3.2: Hash Data Structures Define GPU-side structures: - **cell_start** array: Index where each cell begins in sorted list - **cell_end** array: Index where each cell ends - **sorted_agents** array: Agent IDs sorted by cell - **agent_cells** array: Which cell each agent belongs to - Allocate CUDA memory for all structures

Task 3.3: Cell Size & Grid Parameters

- Cell size = perception_radius (most efficient)
- Calculate grid dimensions based on world bounds
- Handle edge cases (world boundaries)
- Support dynamic grid resizing if world changes

Task 3.4: Hash Function Implementation

- Implement 2D hash function:
 - Convert world position to grid coordinates (x_cell, y_cell)
 - Hash (x, y) to bucket index
 - Common: $(x * \text{prime1}) \text{ XOR } (y * \text{prime2}) \bmod \text{table_size}$
- Must be deterministic and well-distributed
- Device function callable from kernels

Task 3.5: Cell Assignment Kernel Write CUDA kernel: - Each thread handles one agent - Read agent position - Calculate grid cell coordinates - Hash to bucket index - Store in `agent_cells[agent_id]` - No synchronization needed (independent writes)

Task 3.6: Parallel Sorting Integration

- Use Thrust library for GPU sorting:
 - Sort `agent_cells` array (keys)
 - Simultaneously permute agent IDs (values)
 - Result: agents grouped by cell in sorted order
- Learn Thrust API (`thrust::sort_by_key`)
- Handle device pointers vs raw CUDA pointers
- Benchmark sorting time

Task 3.7: Cell Boundary Detection Kernel Write kernel to find cell starts/ends: - Each thread examines one agent in sorted array - Check if `sorted_cells[i] != sorted_cells[i-1]` - If true: this is start of a new

cell - Mark `cell_start[cell_id] = i` - Check if `sorted_cells[i] != sorted_cells[i+1]` - If true: this is end of a cell - Mark `cell_end[cell_id] = i+1` - Handle edge cases (first/last agent)

Task 3.8: Neighbor Query Function Provide device function for Person 2 to use:

Input: `agent_index, agent_position`

Output: list of neighbor agent indices (or callback approach)

Algorithm:

1. Calculate my grid cell
2. Loop over 9 neighboring cells (3x3 in 2D):
 - Get cell hash
 - Lookup `cell_start[hash]` and `cell_end[hash]`
 - Iterate agents in that range
 - Perform distance check
 - Add to neighbor list if within perception radius

Task 3.9: Hash Collision Handling

- Understand that collisions are OK (multiple cells $\rightarrow$ same bucket)
- Verify all neighbors are still found
- Test with pathological cases (all agents in one cell)
- Document collision rate

Task 3.10: Optimization

- Minimize atomic operations
- Use shared memory for cell data if beneficial
- Profile memory access patterns
- Compare performance: naive $O(n^2)$ vs hashed $O(n)$
- Document speedup factor

Task 3.11: Dynamic Updates

- Rebuild hash every frame (positions change)
- Pipeline: assign $\rightarrow$ sort $\rightarrow$ find boundaries $\rightarrow$ query
- Measure total hash construction time
- Keep rebuild time $<$ simulation time

Task 3.12: Visualization Support (for Person 1)

- Provide debug data: which cells are populated
- Cell occupancy counts (for heat map)
- Export grid boundaries for rendering

Files Person 3 Owns:

src/spatial_hash.cu/h
src/hash_kernels.cu
src/neighbor_query.cuh

Integration Points:

- Person 2 calls hash build before each simulation step
 - Person 2 queries neighbors during force calculation
 - Must agree on data structure layout
 - Exposes:
 - void buildSpatialHash(Agent* agents, int count)
 - Device function for neighbor queries
-

PERSON 4: UI, FEATURES & INTEGRATION ENGINEER

Role: User interface, game logic, and feature implementation

Dependencies Needed:

- ImGui (UI library)
- GLFW (for UI rendering context)
- Access to simulation parameters
- Understanding of overall system

Input Requirements from Others:

- **From Person 2:** Ability to set simulation parameters
- **From Person 1:** Ability to set rendering options
- **From Person 3:** Performance metrics

Output Deliverables:

- Fully functional UI with all controls
- Predator-prey system logic
- Obstacle system
- Preset configurations
- Complete documentation

Detailed Tasks:**Task 4.1: ImGui Integration**

- Initialize ImGui with OpenGL backend
- Create ImGui context in main loop

- Set up rendering (after scene, before buffer swap)
- Handle ImGui input (mouse/keyboard don't conflict with camera)
- Choose UI style/theme

Task 4.2: Main Control Panel Design and implement UI window with: - **Agent count slider:** - Range: 1,000 to 2,000,000 - Logarithmic scale - On change: trigger agent reinitialization

- **Boids parameter sliders:**
 - Separation weight (0-5)
 - Alignment weight (0-5)
 - Cohesion weight (0-5)
 - Perception radius (10-200)
 - Max speed (0.5-10)
 - Max force (0.1-2)
- **Visual settings:**
 - Color scheme dropdown
 - Trail length slider
 - Agent size slider
 - Camera FOV (for 2.5D)
- **Performance display:**
 - FPS counter
 - Frame time (ms)
 - Simulation time vs render time breakdown
 - Memory usage

Task 4.3: Simulation Controls

- Play/Pause button
- Reset button (reinitialize all agents)
- Step-by-step mode (advance one frame)
- Time scale slider (slow motion / fast forward)
- Save/load simulation state

Task 4.4: Preset System Create preset configurations: - **“Bird Flock”**: Moderate cohesion, high alignment - **“Fish School”**: Tight cohesion, moderate alignment - **“Chaos”**: Low cohesion, high separation - **“Orbiting”**: Special parameters for vortex formation - Each preset loads specific parameter values - Store presets in JSON config files - Allow user to save custom presets

Task 4.5: Agent Type System Design multi-type agent logic: - **Agent type enum:** PREY, PREDATOR, NEUTRAL - **Type-specific parameters:** - Different max speeds - Different colors (coordinate with Person 1) - Different behavioral rules

- **UI controls:**

- Percentage sliders for each type
- “Add Predator” button (spawn at random location)
- “Add Prey” button
- “Convert Random to Predator”

Task 4.6: Predator-Prey Logic Design Work with Person 2 to implement:

- **Predator behavior specifications:** - Chase nearest prey within detection range - Faster than prey (1.5x speed multiplier) - “Eat” prey within close range (remove prey agent) - Energy system (depletes over time, recharged by eating)
- Die if energy reaches zero

- **Prey modifications:**
 - Fear response: extra separation from predators
 - Detection range for predators > normal perception
 - Panic speed boost when predator very close
- **Balance parameters** to make interesting dynamics

Task 4.7: Obstacle System Design obstacle mechanics: - **Obstacle struct:**

- Type: circle, rectangle, line - Position, size - Static or moving

- **UI for obstacles:**
 - List of current obstacles
 - Add/remove buttons
 - Click to place obstacle mode
 - Drag to move obstacles
- **Collision logic** (work with Person 2):
 - Ray-cast ahead of agent
 - Calculate avoidance force
 - Specify avoidance weight parameter

Task 4.8: Environmental Effects

- **Wind/current fields:**
 - Vector field affecting all agents
 - UI: direction and strength sliders
 - Optional: Perlin noise for turbulence
- **Attraction/repulsion zones:**
 - Click to place attractor point
 - Agents steer toward/away
 - Falloff radius

Task 4.9: Statistics & Analytics Display real-time statistics: - Average velocity - Average neighbor count - Density heat map - Predator/prey population over time (graph) - Spatial distribution metrics

Task 4.10: Scenario System Create demo scenarios: - **“Murmuration”**: Load preset + auto-camera movement - **“Predator Attack”**: Spawn predator in center of flock - **“Obstacle Course”**: Spawn obstacles, watch avoidance - **“Migration”**: Moving attraction point - Scripted camera paths for demos

Task 4.11: Export & Recording

- Screenshot button (save current frame as PNG)
- Record button (save sequence of frames)
- Export simulation state to file
- Load simulation state from file
- Parameter export to JSON

Task 4.12: Documentation Create comprehensive docs: - **README.md**:
- What the project does - Build instructions - Controls guide - Parameter explanations

- **USER_MANUAL.md**:
 - How to use each feature
 - Tips for interesting patterns
 - Parameter tuning guide
- **ARCHITECTURE.md**:
 - Code organization
 - Module responsibilities
 - Data flow diagrams
- **API.md**:
 - Function documentation
 - Integration points between modules

Task 4.13: Demo Preparation

- Create demo script (order of features to show)
- Prepare 3-5 impressive scenarios
- Record demo video (screen capture)
- Create presentation slides
- Performance benchmark charts

Files Person 4 Owns:

```
src/ui.cpp/h
src/presets.cpp/h
src/obstacles.cpp/h
src/scenarios.cpp/h
config/presets.json
docs/README.md
docs/USER_MANUAL.md
docs/ARCHITECTURE.md
```

Integration Points:

- Passes parameters to Person 2's simulation
 - Requests rendering options from Person 1
 - Displays metrics from Person 3
 - Coordinates main loop timing
-

SHARED RESPONSIBILITIES**Everyone:**

- Document their code
- Profile performance in their domain
- Help debug integration issues

Shared Files:

src/main.cpp (all contribute)
src/agent.h (agreed upon by all)
CMakeLists.txt (Person 1 initially, others add to)

Communication Protocol:

- Use shared header files for interfaces
 - Document function contracts clearly
 - Notify team before changing shared structures
 - Merge to main only when module passes tests
-

Timeline Synchronization**Week 1 (All together)**

- Days 1-2: Individual setup
- Days 3-5: Work in pairs on CPU baseline

Week 2 (Parallel work begins)

- Person 1: Focus on rendering system
- Person 2: CUDA naive implementation
- Person 3: Hash structure design + cell assignment
- Person 4: Basic UI + parameters

Integration Point 1 (End of Week 2):

- Merge Person 1 + Person 2 → GPU-rendered simulation

Week 3 (Advanced features)

- Person 1: Visual effects
- Person 2: Integrate Person 3's hash
- Person 3: Complete neighbor queries
- Person 4: Predator-prey + obstacles

Integration Point 2 (Mid Week 3):

- Merge Person 3 → Performance boost

Integration Point 3 (End Week 3):

- Merge Person 4 → Complete feature set

Week 4 (Polish)

- All: Bug fixes, optimization, documentation
 - Person 1: 2.5D upgrade
 - Person 2: Final performance tuning
 - Person 3: Benchmarks
 - Person 4: Demo preparation
-